ConfigStream

Scalable Configuration Change Storage and Streaming Analytics

Phạm Công Hoàng Lê Tiến Hợp
Mai Vũ Hoàng Nguyễn Đức Dũng
Group 4 · Big Data Storage & Processing

First milestone checkpoint · 26 October 2026

The problem: configuration changes over time

Router configuration: version 10

```
hostname router-001
logging enabled
telnet disabled
```

Version 11

```
hostname router-001
logging disabled
telnet enabled
```

What changed, when, and on which asset?

A current configuration file alone cannot answer all three.

Four datasets, four meanings of change

Terraform / TerraDS: versions

Infrastructure code stored in Git.

v1 → **v2**: which lines changed?

RPKI: observed daily states

Observed route-authorization records.

Day 1 → **day 2**: which members differ?

RIS: routing events

BGP announcements and withdrawals.

Announce → **withdraw**: what happened?

DNS: measurements

Recorded DNS query and response data.

Name → **IP**: what was observed, when?

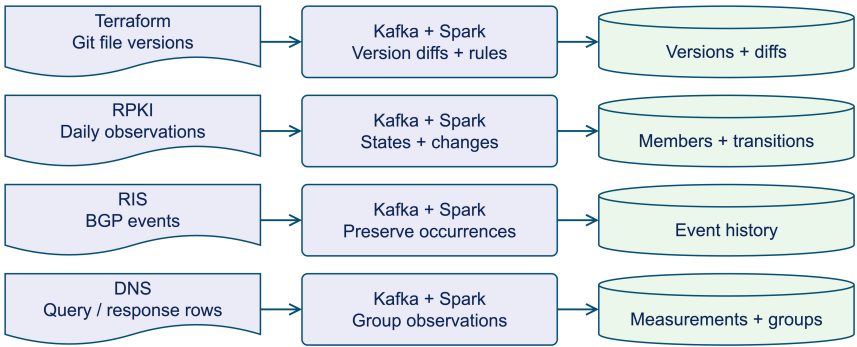
Prepared data: accepted scope for each dataset

Dataset	Accepted amount	Selected scope
Terraform	**10,188** Git versions	6,505 file histories / 375 repositories
RPKI	**14** complete daily observations	ripencc.tal, 1–14 Sep 2026
RIS	**4,725,156** event occurrences	rrc00, 1 Sep 2026, 00:00–01:00 UTC
DNS	**2,234,863** raw measurements	Two .li source objects, 1–2 Sep 2026

Controlled synthetic inputs also test retries and late arrivals.

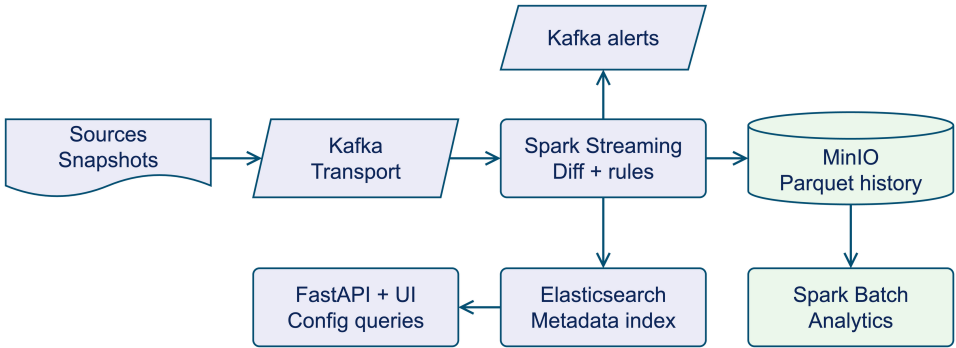
Source manifests and result receipts are versioned in Git.

System architecture: four separate processing paths



Kafka + Spark + MinIO (Parquet), with separate schemas and processing.
Config search: Elasticsearch + FastAPI. Dashboard: accepted results from all four.

Configuration path: transport, compute, store, serve



Lake writes complete before search indexing and alert publication.
Search can be rebuilt from committed Parquet; batch analytics reads the lake.

Input contract: identity, source version and two clocks

Field	Illustrative value	Role in processing
`asset_id`	`router-001`	Groups one configuration history
`snapshot_id`	`router-001-v11`	Detects conflicting reuse of an ID
`version`	11	Selects the adjacent predecessor, v10
`event_time`	08:00 UTC	Source time for watermark and timers
`ingest_time`	08:03 UTC	Collector acceptance time
`hash`	SHA-256 of content	Checks inline content integrity

The schema accepts exactly one of `content` or `content_uri`.
The current diff processor handles inline text; URI content is not fetched.

Kafka: delivery position differs from source version

Three input topics

`config.assets`: metadata

`config.events`: change audit

`config.snapshots`: content

Key: `asset_id`

Spark processes the snapshots topic.

Example: one snapshot-topic partition

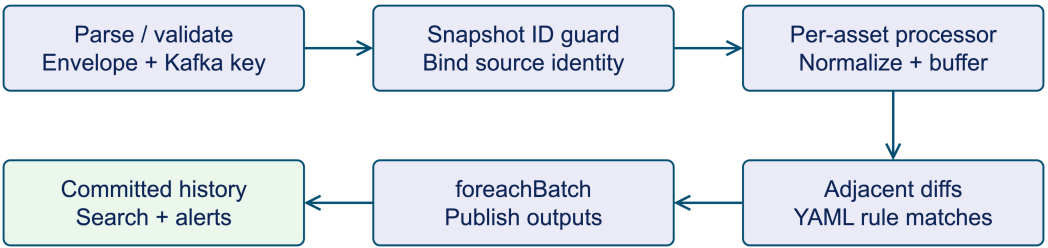
Offset	40	41	42
Asset	R1	R1	R1
Version	v1	v3	v2

Offset 42 arrives later, but v2 belongs between v1 and v3 in source history.

Why Kafka and state are both needed

Kafka retains a replayable log per partition. Spark reconstructs version relationships. Checkpointed progress resumes processing while source offsets remain available.

Spark: two stateful stages before output publication



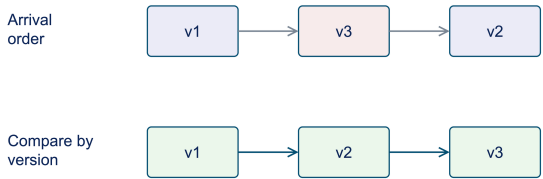
Key 1: `snapshot_id`

Validate content hash and source binding. Same binding: suppress retry. Conflict: diagnostic.

Key 2: `asset_id`

Retain normalized versions + emitted-edge flags.
Event-time timers and a version cap bound state.

Version state: buffer gaps and emit each adjacent pair



Input	Retained versions	New output
v1	{1}	Baseline only; no predecessor
v3	{1, 3}	Hold the gap; no 1 → 3 comparison
v2	{1, 2, 3}	Emit 1 → 2 and 2 → 3; mark both edges
Retry v2	Unchanged	No new logical diff while identity/state is retained

From normalized text to an explainable rule alert

Diff: version 10 → 11

```
- logging enabled
- telnet disabled
+ logging disabled
+ telnet enabled
```

Normalization handles selected text noise and masks supported secret patterns.

Actual YAML rule

```
id: telnet-enabled
severity: critical
operation: added
pattern: "^telnet enabled$"
```

An added line matches the regex.

The alert links the asset, snapshot and rule.

Logging disabled: a separate **high** rule.

Line comparison and explicit rules; alerts are not verified security incidents.

MinIO: raw evidence and typed Parquet history

```
config-lake/
  raw/snapshots/
  processed/snapshots/
  processed/diffs/
  processed/alerts/
  processed/diagnostics/
  _commits/
  analytics/
```

Raw: binary key/value plus Kafka topic, partition, offset and timestamp.

Processed: typed normalized snapshots, diffs, alerts and diagnostics; Snappy Parquet.

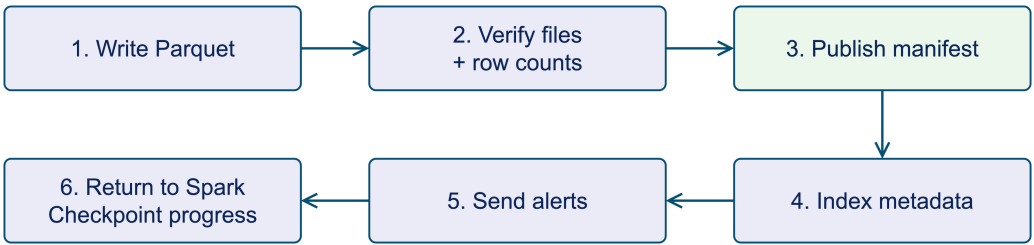
Namespace: query ID + batch ID.
Date folders: UTC year/month/day.
Snapshot date uses source event time.

```
processed/snapshots/query_id=.../batch_id=.../year=2026/month=10/day=5/
```

Three different meanings of partition

Kafka: a log shard. Spark: execution tasks. Lake: date-based file paths.

Commit protocol: publish verified history before serving



Manifest binds physical output

File paths, byte sizes, SHA-256 and counts.
Reader applies the typed dataset schema.

Retry checks the full typed multiset

Values and row multiplicity must match.
Equal row counts alone are insufficient.

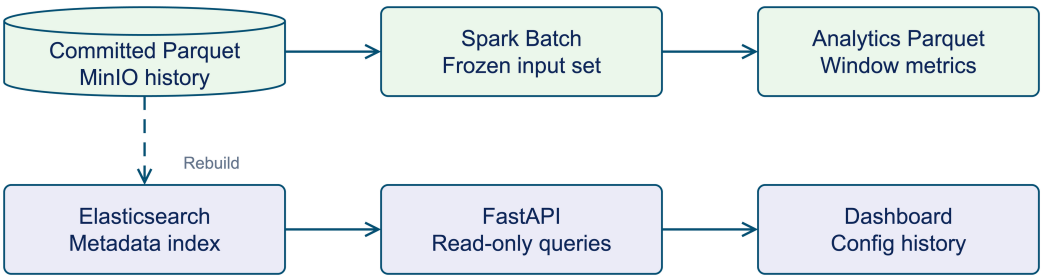
Failure recovery: what happens if a batch stops midway?

Failure point	Visible state	Retry behavior
Before lake manifest	Partial files are not committed history	Readers exclude the incomplete batch
After commit, during search	Lake is valid; search may be incomplete	Verify/reuse lake; index deterministic document IDs again
After alerts, before progress commit	Some alerts may already be delivered	Reprocessing may append physical alert duplicates

Spark checkpoint: source progress and operator state.

Lake manifest: which verified data files are committed.

Serving: historical analytics and metadata queries



Batch: freeze input manifests; aggregate UTC windows, alert trends and prior-window flags.

API: filter asset/time; latest snapshot by version; cursor pagination. No raw config bodies.

Progress: runnable code and a recorded pipeline replay

Repository

`generator/`	input data
`processing/`	diff and rules
`spark/`	streaming jobs
`api/`	read API
`frontend/`	dashboard

Small demo, no services required

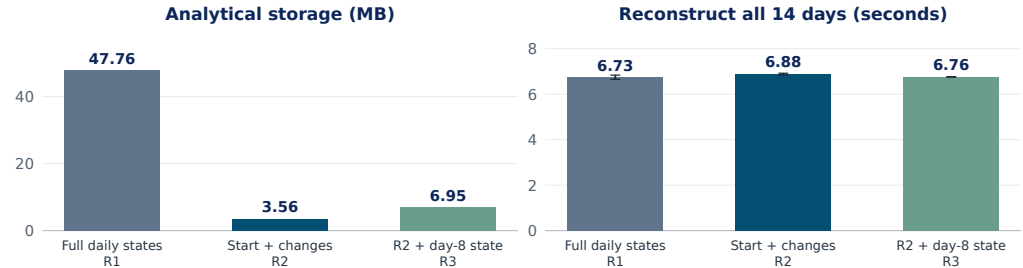
```
make setup
make processing-demo
```

178 core tests passed
State logic, diffs and rules
Rechecked on 25 October 2026

Recorded TerraDS replay: 10,188 snapshots → 3,683 diffs; 29.4 MB Parquet.

RPKI: less storage, similar reconstruction time

Same 14 daily observations. All three layouts reconstruct the exact same states.

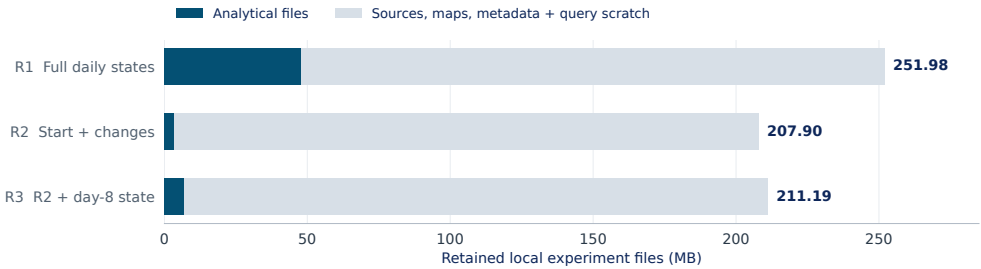


R2 saves 92.6% of analytical storage.

Reconstructing all 14 days still takes about 6.9 seconds.

The storage saving depends on what we count

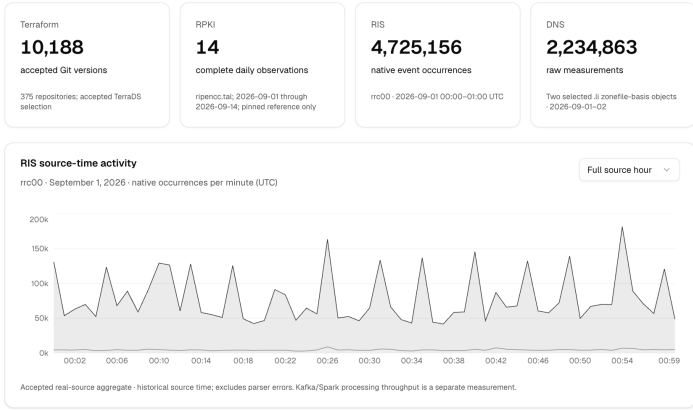
Same RPKI experiment, now including the other retained files.



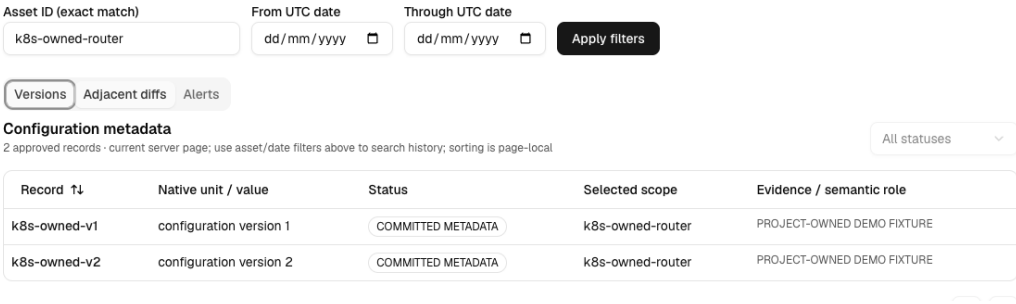
R2 saves 17.5% across measured local experiment files.

Sources, provenance maps and query scratch still occupy most of the space.

Dashboard: four domains in one evidence overview



Dashboard: inspect configuration history by asset



1. **Filter** by asset and UTC dates.
2. **Select** versions, adjacent diffs or alerts.
3. **Inspect** safe metadata from the configuration API.

Checkpoint readiness and next milestone

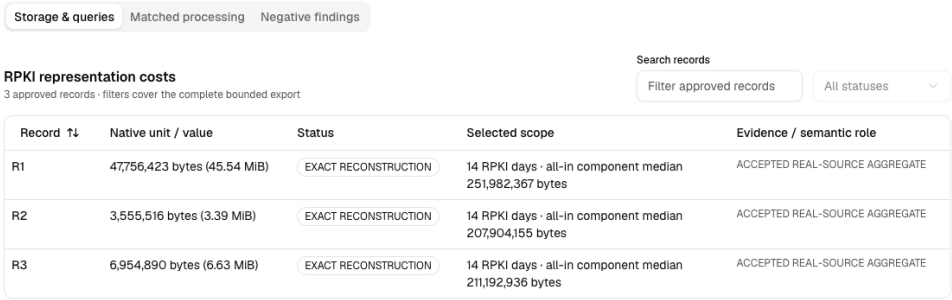
Requirement	What the group can show
Executable code	Runnable demo, domain pipelines and tests
Prepared data	Terraform, RPKI, RIS, DNS and source manifests
Architecture	Kafka, stateful Spark, committed lake and serving

Next: rehearse the latest integrated demo, repeat recovery checks, and extend workload measurements.

Existing deployment evidence is local / single-node. Scaling and HA remain open.

Thank you — questions?

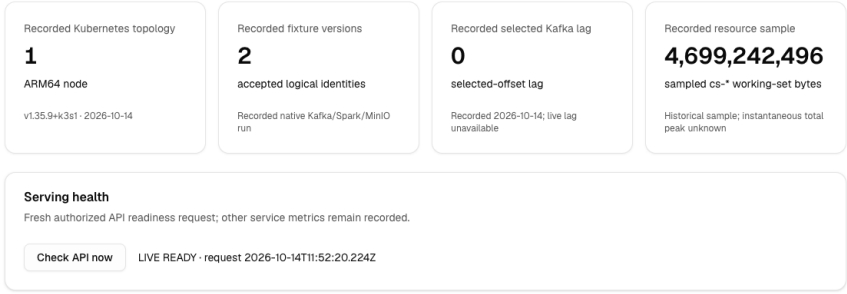
Dashboard: inspect the evidence behind a comparison



The experiment view exposes storage, query measurements and evidence references.

Compare each number within its declared workload and measurement boundary.

Dashboard: distinguish recorded runs from an API probe



Top cards: recorded Kubernetes, lag and resource evidence.

Check API now: a fresh readiness request when the backend is available.

The READY label in this historical capture applies only to its displayed timestamp.